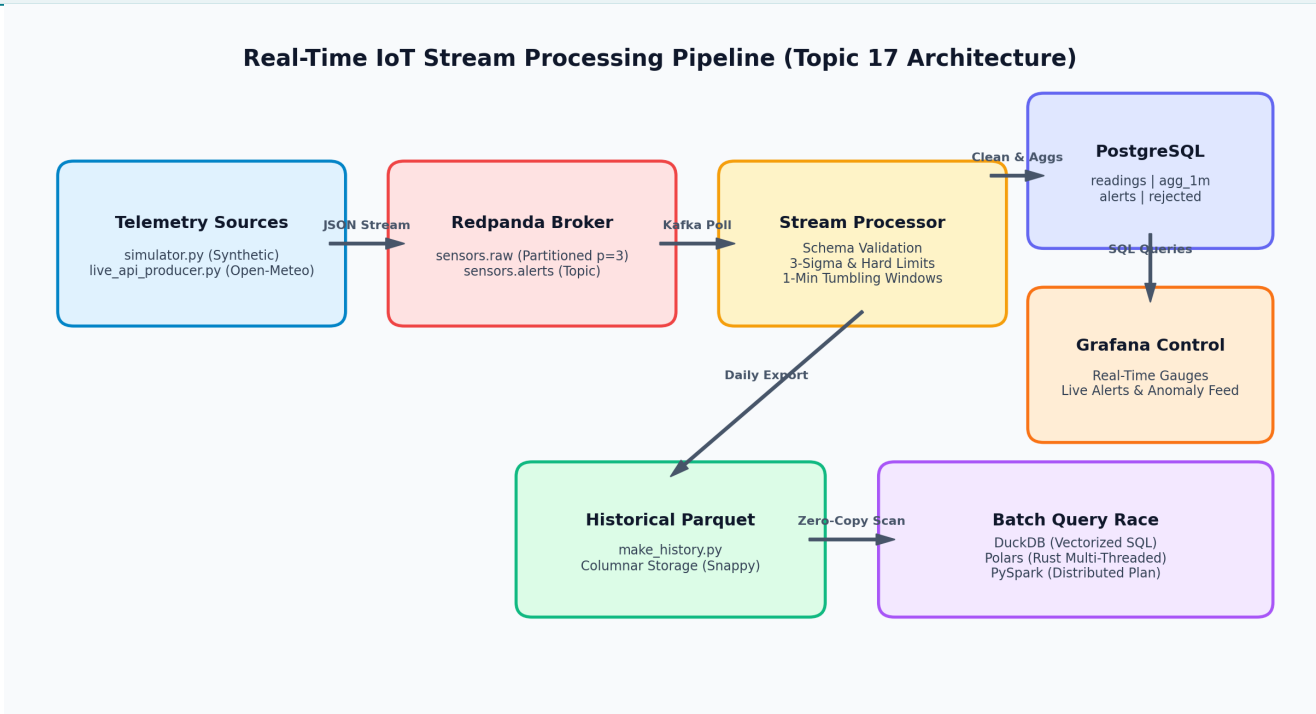
A BEGINNER-FRIENDLY PROJECT GUIDE

Real-Time IoT Stream Processing

What the project is, what each part does, and how a reading travels through the whole system

Think of it as a small digital control room for building sensors. This guide walks from the first sensor message to the dashboard, then explains historical analytics and deployment.



Prepared from the project source code, configuration, and documentation. 13 October 2026.

1. The project in one minute

This project demonstrates how a smart building could collect readings from sensors, spot unusual conditions, save useful records, and show the latest situation to an operator. It combines a message broker, a Python processing program, a database, a dashboard, and a separate historical analytics path.

Imagine a delivery system: sensors write parcels, Redpanda is the moving belt, the Python processor inspects each parcel, PostgreSQL stores sorted records, and Grafana is the wall display. Historical Parquet files are a saved archive that different analytics tools can read later.

The main question it answers

How can incoming readings be handled quickly and consistently, while bad messages are set aside and unusual values are made visible? The project is a learning and demonstration pipeline. It is not a finished commercial building-monitoring product.

What happens in order

Step	In everyday words	Project part
1. Create a reading	A sensor or simulator makes a small record with a device, time, temperature, and power.	simulator.py or live_api_producer.py
2. Send it	The record is sent to a named stream where programs can read it.	Redpanda topic sensors.raw
3. Inspect it	The processor checks the data, flags unusual values, and groups readings by minute.	processor.py
4. Save it	Clean readings, alerts, rejects, and summaries are stored in separate database tables.	PostgreSQL
5. Show it	Dashboard queries read the database and display totals, trends, and recent issues.	Grafana
6. Study history	A saved Parquet dataset is queried by multiple analytics engines.	DuckDB, Polars, PySpark example

2. The problem it models

A building may have many rooms and devices. If readings are only printed to a terminal, an operator cannot easily see trends, find malformed input, or identify an extreme temperature. This design gives incoming data a path through checks, storage, and visualization.

Example: if one sensor reports 58 degrees, the system can retain that value as a valid numeric reading while also recording an anomaly alert. If a message is missing its temperature field or contains invalid JSON, it can go into a rejected-record log with a reason.

3. The main pieces

Each technology has one job. Keeping the jobs separate makes it easier to replace or scale a part later.

Part	What it is	Why it is here
Sensor simulator	A Python program that invents plausible device readings.	Lets the system be exercised without physical sensors. It can also inject spikes and malformed messages.
Open-Meteo producer	A Python script that requests current weather from a public API.	Adds a real external data source. It maps weather values into the project message shape.
Redpanda	A message broker that stores streams of events in topics. It speaks the Kafka protocol.	Separates producers from consumers. The producer can publish while the processor reads independently.
Stream processor	A Python Kafka consumer with validation, anomaly checks, aggregation, and database writes.	Turns raw messages into clean records, alerts, rejected rows, and minute summaries.
PostgreSQL	A relational database with named tables and SQL queries.	Holds recent processed data so dashboards and people can query it.
Grafana	A dashboard application that runs SQL queries and draws panels.	Turns stored numbers into totals, charts, and recent-event tables.
Parquet	A compressed column-oriented file format.	Stores historical readings compactly for analytical scans.
DuckDB and Polars	Two local analytical query engines.	Run the same questions against Parquet so their timings and outputs can be compared.
Apache Spark	A distributed data-processing framework, with a DataFrame API.	The repository includes a PySpark analytics example, configured for local machine mode.
Kubernetes	A system for running containerized applications as pods and services.	Runs the broker, database, processor, simulator, Console, and dashboard together.

4. What is a message?

A message is one small event encoded as JSON. JSON is text with named fields and values, so different programs can exchange a predictable structure. A typical simulator reading looks like this:

```
{
"device_id": "sensor-01",
"room": "Room-201",
"event_ts": "2026-10-13T12:00:00+00:00",
"temperature_c": 22.5,
"power_kw": 1.2
}
```

Field	Meaning
device_id	Which sensor created this reading.
room	Where the sensor is assigned.
event_ts	When the reading was produced, in UTC time.
temperature_c	Temperature in degrees Celsius.
power_kw	Power draw in kilowatts.

A stable field contract matters: both the sender and receiver need to agree on field names, data types, and meanings. A missing or unusable field means the processor cannot reliably interpret that message.

5. Follow one reading through the system

This section follows one normal temperature reading. The same path also explains what changes when the message is bad or unusual.

Step 1: the source creates a reading

simulator.py keeps a small state for each virtual device. Each cycle it adjusts temperature and power slightly, adds the current timestamp, and makes a JSON record. The simulator can be configured with options that inject high temperature/power spikes or broken rows. If the broker is unreachable, its code can switch to mock mode and print messages to the console; those printed messages are not delivered to Redpanda.

Step 2: Redpanda accepts the event

The simulator publishes the record to the sensors.raw topic. A topic is a named stream, similar to a mailbox or conveyor lane. Redpanda stores the event so a consumer can read it. The project also creates sensors.alerts for alert events. Kafka-compatible clients communicate with Redpanda using broker addresses and topic names.

Step 3: the processor reads and validates it

processor.py is the consumer. It continuously polls the raw topic. It attempts to parse the message as JSON and check required fields and values. If parsing or validation fails, the processor records the original payload and a reason in rejected. If valid, the record can continue to clean-reading storage.

Step 4: check whether the value is unusual

There are two kinds of anomaly check in the implementation. A fixed limit flags temperature above 45 C or power above 10 kW. A dynamic check compares a reading to recent history for the same device: after enough baseline readings are available, it flags values more than three standard deviations from the recent average, when the variation is large enough to make that rule meaningful. The code keeps up to 20 recent values per device in memory.

An alert is additional information about a reading. It does not mean that all high readings are discarded. The alert is written to the alerts table and published to sensors.alerts, while valid clean readings are written to readings.

Step 5: form one-minute summaries

The TumblingWindowAggregator groups values by device and by fixed one-minute time bucket. For each closed bucket it computes average temperature, maximum temperature, average power, and the number of readings. These summaries are written to agg_1m. A tumbling window does not overlap the next one: each event timestamp belongs to one minute bucket.

Step 6: save information in PostgreSQL

The database is the queryable store. The schema defines four tables: readings for valid sensor events; agg_1m for per-device minute summaries; alerts for anomaly events; and rejected for invalid input plus an explanation. Indexes are created on event times and device/time to help common dashboard queries.

6. How the dashboard works

Grafana does not usually collect the sensor messages itself in this project. It asks PostgreSQL questions using SQL, then shows the results in panels. The provisioned dashboard refreshes frequently, so new rows can appear without an operator manually rerunning queries.

Panel	Question it asks	How to read it
Total Clean Readings	How many rows exist in readings?	A rising total means valid events have been stored over time.
Throughput (Last 1m)	How many reading timestamps fall in the last minute?	A rough recent activity count. It is not the same as a broker-acknowledged benchmark.
Active Anomaly Alerts	How many rows are in alerts?	A total alert count; the panel name says active, but the query counts all saved alerts.
Rejected Bad Rows	How many rows are in rejected?	Shows validation failures collected so far.
Live Temperature / Power	What values were stored inside the selected dashboard time range?	Time-series charts group points by room. Empty charts can mean there are no readings in the selected time range.
Recent alert/reject logs	What are the newest records?	Tables show a small recent sample for inspection.

Why might a panel say No data?

- No simulator is currently publishing, or the processor is not consuming.
- The processor fell back to mock storage because it could not reach PostgreSQL.
- The selected Grafana time range does not include the event timestamps.
- A panel query or data-source connection has a configuration problem.

The provided dashboard preview graphic is generated artwork for documentation, not a captured image of a live Grafana screen. The actual dashboard configuration lives in the Grafana provisioning files.

7. The historical analytics path

The live stream and historical analytics solve different needs. The live path handles events as they arrive. The historical path stores a larger set in a Parquet file and answers questions over that saved data. The project can generate synthetic history or export rows from PostgreSQL to Parquet.

Parquet stores related values in columns. For example, a query that needs only temperature and timestamp can scan those columns without reading every field. Compression can also reduce disk use.

The project runs four example questions

- For each room and hour: average and maximum temperature.
- For each room and day: total power consumption.
- Which devices have the most readings beyond the anomaly thresholds?
- How many records exist for each device?

DuckDB expresses these questions as SQL. Polars expresses them as dataframe operations. The benchmark runs each query multiple times and records a median runtime, which is less affected by one unusually slow or fast run. The saved

comparison is local to one machine and one generated dataset; it does not compare large distributed clusters.

8. The Spark example and performance ideas

Spark is designed to process data in parallel across CPU cores or across a cluster of machines. The repository has spark_analytics.py, which reads Parquet into a Spark DataFrame and demonstrates filters, derived columns, grouping, repartitioning, caching, and actions.

Spark word	Plain meaning in this project
DataFrame	A table-like collection of rows that Spark can process.
Transformation	A plan for changing or grouping data. Spark can wait to execute it until a result is needed.
Action	A command such as count() that asks Spark to actually produce a result.
Partition	A chunk of data that can be worked on in parallel.
Shuffle	Data movement between partitions, often required before groups or joins. Shuffling can be expensive.
Repartition	Redistribute rows into a chosen number of partitions. Here the code repartitions to four by device_id.
Cache / persist	Keep a computed dataset in memory for reuse, instead of rereading/recomputing it.
Shuffle partition setting	Controls the number of partitions Spark uses for certain SQL operations. The example sets it to four.

Important: this script selects local[*]. That means Spark uses available cores on one computer. It is not evidence that a multi-machine Spark cluster or a Hadoop cluster has been deployed. The code can fall back to a Polars-style emulation if Spark cannot start, such as when Java is unavailable.

A cache optimization only helps if the cached data is reused and the saved work exceeds the cost of building the cache. A partition count that is too small may leave cores idle; one that is too large may create scheduling and file overhead. Performance tuning should compare measured runs on the target data and hardware. The repository includes the mechanisms, but a measured Spark cache-versus-uncached result is not included in the saved result table.

9. Running the services in containers and Kubernetes

A container packages an application with its dependencies so it can run in a predictable environment. Docker Compose describes several related containers on one machine. Kubernetes runs containers inside pods and provides stable service names so one component can find another.

The Kubernetes manifests define the iot-pipeline namespace and the core workloads: Redpanda, PostgreSQL, the processor, the simulator, Redpanda Console, and Grafana. The in-cluster simulator publishes to Redpanda by its Kubernetes service name. PostgreSQL has a persistent volume claim so database files can survive an individual pod restart, subject to the cluster storage setup.

Kubernetes concept	Simple explanation
Namespace	A named area that groups this project’s resources.
Pod	The running unit that contains one or more containers.
Deployment	Tells Kubernetes how many copies of a service to run and how to replace them.
Service	A stable network address used by other pods or by port forwarding.

Kubernetes concept	Simple explanation
ConfigMap	Configuration or script content mounted into a pod.
PersistentVolumeClaim	A request for storage that outlives a database pod restart.
Port forward	A temporary tunnel from a local port to a service inside the cluster.

The Kubernetes deployment is for the streaming stack. Spark is not deployed there in the current manifests. Local Kubernetes, like Docker Desktop Kubernetes, Minikube, or Kind, is useful for learning, but one local broker pod is not the same as a production, fault-tolerant multi-node cluster.

10. What the recorded results mean

The saved DuckDB/Polars comparison reports that Polars ran faster for each of the four recorded query timings. This is a result for that run, dataset, and computer, not a universal ranking. Re-run it with the same input and machine details before using it as a broad performance claim.

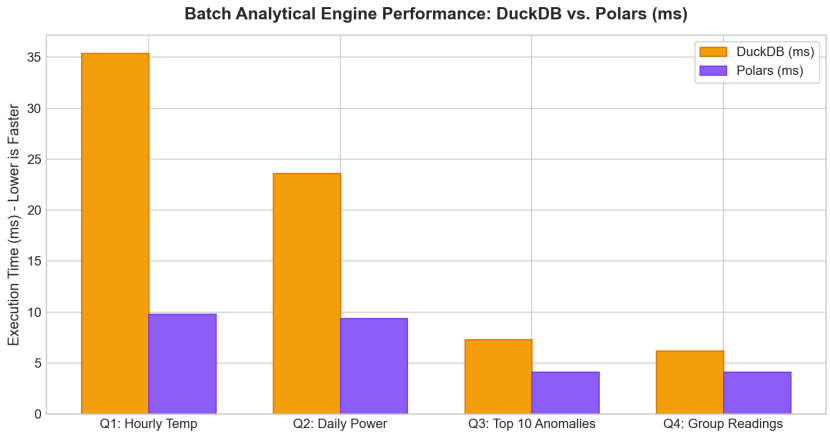


Figure 2. Generated chart based on the saved local engine timing table.

The saved streaming throughput table and failure-recovery summary need careful labeling. capture_results.py calls the simulator with mock_mode=True, so its throughput values time a mock publish path, not confirmed Redpanda delivery. failure_test.py uses an in-memory list and a simulated offset variable; it does not crash and restart a live Kafka consumer. These are code demonstrations, not broker benchmarks or zero-loss guarantees.

11. How to try the project

The repository documents both Docker Compose and Kubernetes deployment. In either case, the general order is: start the services; wait until the broker and database are ready; run or deploy the processor; run the simulator; then open Grafana. Redpanda Console is useful for browsing topics. The Kubernetes README contains the full command sequence and port-forward commands.

When testing, inject anomalies or bad rows in a controlled run, then inspect the alerts and rejected tables. For batch analytics, generate a Parquet dataset, then run the DuckDB/Polars comparison. For a live API demonstration, run the API producer while the broker and processor are reachable. If messages switch to mock mode, they are printed locally rather than passed through the full pipeline.

12. Strengths and current boundaries

What is present	What this means / boundary
End-to-end streaming design	The repository connects event production, broker transport, processing, SQL storage, and a dashboard.
Malformed and anomaly examples	The simulator supports test inputs that demonstrate rejection and alert logic.
API source	Open-Meteo current weather can be mapped into the message shape. Power is calculated as a proxy, not measured building power.
Historical query comparison	DuckDB and Polars run the same four types of query over Parquet. Timings are single-machine measurements.
Kubernetes manifests	Core streaming services are represented as Kubernetes resources. Spark is not part of that Kubernetes deployment.
Spark code	The analytics example uses local[*], so it demonstrates local parallel Spark, not a multi-node cluster.
Mock result paths	Throughput and failure outputs should not be described as real broker tests.
Dashboard evidence	The dashboard JSON is real configuration; the bundled preview image is a mock illustration.

13. Small glossary

Term	Meaning
API	A way for one program to request data or a service from another program.
Broker	A service that accepts, stores, and delivers messages between producers and consumers.
Consumer	A program that reads events from a broker topic.
Producer	A program that sends events to a broker topic.
Event time	The timestamp carried by the reading, usually when the reading was made.
JSON	A common text format for structured data with named fields.
Parquet	A compressed file format optimized for storing and analyzing tables by column.
Schema	The agreed fields, types, and meaning of data.
SQL	A language for asking questions of relational databases.
Topic	A named stream of messages in Redpanda/Kafka.
UTC	A worldwide reference time standard that avoids local time-zone ambiguity.
Window	A time bucket used to group multiple events, such as one minute of readings.

14. The whole idea, summarized

A sensor reading starts as a small JSON record. Redpanda carries it. A Python processor checks it, stores clean data, creates anomaly alerts, and calculates minute summaries. PostgreSQL keeps those results queryable. Grafana asks the database questions and shows an operator the answers. Separately, Parquet holds historical data so DuckDB, Polars, and the Spark example can analyze it. Kubernetes provides a way to run the streaming services together.

The most important way to describe the project accurately is as a working educational pipeline with a Kubernetes streaming deployment, a local Spark analytics example, and local historical engine comparisons. Some result artifacts are mocked, so they should be described as demonstrations until they are rerun against the actual broker and services.

Project files this guide describes

README.md; processor.py; simulator.py; live_api_producer.py; schema.sql; analytics.py; make_history.py; spark_analytics.py; capture_results.py; failure_test.py; docker-compose.yml; the k8s/ manifests; and Grafana provisioning files.